

Programs Need Collections and Decisions

In Week 1, one variable stored one value:



CORE CONCEPT

Lists Store an Ordered Sequence

A list is an ordered, changeable collection of values. Each value is an element. Its position is its index. Python starts indexing at zero:

`visitorcounts[0]` gives the first value:
120.

`visitorcounts[2]` gives the third value:
128.

`visitorcounts[-1]` gives the final value:
128.

`visitorcounts[0:2]` gives indexes 0 and 1:
[120, 135].

`len(visitorcounts)` gives the number of
elements: 3.

RUN IT AND INSPECT THE RESULT

Lists Can Be Changed

```
model_features = ["age", "income"]  
  
model_features.append("education")  
model_features[1] = "monthly_income"  
  
print(model_features)
```

OUTPUT

```
['age', 'monthly_income', 'education']
```

The same variable still refers to the changed list.

REASONING SEQUENCE

Dictionaries Connect Keys to Values

Frame

A dictionary stores key-value pairs.

1

Transform

A key is the label used to find a value.

2

Validate

Here "age" is a key and 0.75 is its value.

3

Explain

Unlike a list, a dictionary is not accessed by a position such as index 0.

4

Start with:

CORE CONCEPT

Tuples Store a Fixed Ordered Record

A tuple is an ordered collection that is usually used as one fixed record. Tuples use parentheses and support indexing: A tuple is immutable, meaning its elements cannot be replaced after creation.

`result[0]` gives "Asha".

`result[1]` gives 85.

`name, score = result` assigns "Asha" to name and 85 to score.

`result[1] = 90` fails because the tuple is immutable.

RUN IT AND INSPECT THE RESULT

Sets Keep Unique Values

```
predictions = ["spam", "spam", "not_spam", "spam"]
unique_labels = set(predictions)

unique_labels.add("spam")

print(len(unique_labels))
print("not_spam" in unique_labels)
```

OUTPUT

```
2
True
```

For two sets, a & b means values present in both sets.

CORE CONCEPT

Conditions Choose Which Code Runs

A condition is an expression that produces True or False. An if statement runs an indented block only when its condition is true: Python uses a colon and indentation to show which statements belong to a branch.

if tests the first condition.

elif tests another condition if earlier branches were false.

else runs when no earlier branch matched.

Test score >= 90 -> False.

Test score >= 70 -> True.

Print "Pass".

REASONING SEQUENCE

For Loops Process Each Element

Frame

A loop repeats a block of code.

1

Transform

A for loop processes each element in a collection:

2

Validate

During each iteration, the loop variable `score` refers to the next element.

3

Explain

An accumulator is a variable that keeps a running result.

4

For scores = [72, 85, 90]: | Iteration | score | total after addition | |---:|---:|---:| | Start | - | 0 | | 1 | 72 | 72 | | 2 | 85 | 157 | | 3 | 90 | 247 | The loop ends after the final element. The total remains available after the loop.

RUN IT AND INSPECT THE RESULT

While Loops Repeat While a Condition Is True

```
attempts = 0

while attempts < 3:
    attempts = attempts + 1
    print(f"Attempt {attempts}")
```

OUTPUT

```
Attempt 1
Attempt 2
Attempt 3
```

The loop body runs three times.

QUALITY GATE

Functions Turn a Solution into Reusable Code



def begins a function definition.



average is the function name.



values is a parameter, a name used inside the function.



[72, 85, 90] is an argument, the actual value supplied when calling it.

QUALITY GATE

Imports and Decomposition Manage Complexity



load scores;



calculate a summary;



assign a result; and



display the report.

GUIDED LAB

Worked Program: Summarize Quiz Scores

01

**the number of
scores**

02

**the average
score**

03

**how many scores are
passes using a
threshold of 70**

04

**Store the scores in
a list**